PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 9

Similarity and Distance 11

Centroid Clustering 25

Index 35

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

Part I

Geometry of Pattern

Similarity and Distance

Inside the Feature Space

The Why

In supervised learning[1], the learning process relies on data samples x paired with their ground truth labels $\tilde{y}$, which allows a model θ to learn a mapping,

$$\begin{aligned}\theta(x) &= \hat{y}, \\ &\approx \tilde{y}.\end{aligned}$$

[1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org

On the concept of truth, $\tilde{y}$, while being called ground truth, is often an approximation of the true label y. It may come from human annotation, measurements, or other sources.

The presence of labels provides a clear signal for learning, as it allows optimization of a loss function that measures the discrepancy between predictions and true labels. The loss then guides the learning process, enabling the model to adjust its parameters along the gradient to minimize this discrepancy:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\hat{y}, \tilde{y}).$$

Facing the absence of labels, we are left with only the data samples[2] x and no explicit signal to guide learning. The methods we will explore in this course are designed to discover and leverage the inherent structure and patterns[3] in the data itself, without relying on external supervision, thus mostly referred to as unsupervised learning.

[2] J. Rowley. The wisdom hierarchy: Representations of the dikw hierarchy. *Journal of Information and Communication Science*, 33(2):163–180, 2007. doi: 10.1177/0165551506070706. Retrieved November 9, 2024

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

In a first step this requires us to,

- define what we mean by pattern in the context of data.
- understand the geometry of data, which includes concepts of distance, similarity, and neighborhood.

Knowledge Pyramid
Data (1) raw signal
→ Information (*banana*) endowed with meaning
→ Knowledge (≠ *coffee*) structured and contextualized information
→ Wisdom or *knowing and appreciating the Why*.

- learn how to engineer data representations that make pattern more accessible.

THE FUNDAMENTAL QUESTION in unsupervised learning is to find pattern, represent it, and utilize it.

Hands On Experience

CONSIDER six data points in two dimensions. Which points seem to belong together? Why do you think that? How many groups do you see? To which group would you assign new points? Draw areas around each group.

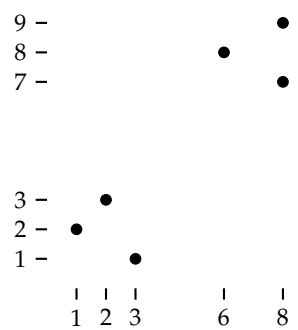

Figure 1: Six points in 2D. Can you see the two clusters? Or are there six? Or maybe one?

WITHOUT LABELS, pattern are ambiguous. With just a single additional point, the pattern can change drastically. Where does the new point belong? Does it belong to the first cluster, the second, or does it form its own cluster? Does this new point change the pattern of the original six points? How would you draw areas around the clusters now? How did that change your assumptions of the first pattern?

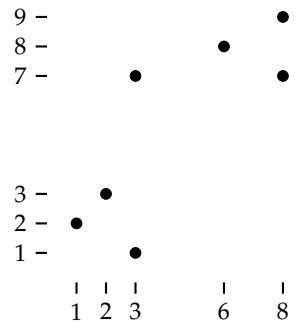

Figure 2: Seven points in 2D. The point at $(3, 7)$ creates ambiguity. Is it part of the first cluster, the second, or its own cluster?

HUMANS ARE REMARABLY GOOD AT FINDING PATTERN IN DATA, even with very little information, and even if it is completely off. It

PAREIDOLY is the tendency to perceive meaningful patterns in random data. Such as animals in clouds.

is a fundamental part of our cognition, and it is what allows us to make sense of the world. However, it also motivates the need for methods to quantify and formalize pattern. On that note it is also good practice to start with a hypothesis of what you expect to find in the data, and then test that hypothesis with the data, rather than just looking for any pattern that may emerge. And on a short managerial note, if the knowledge you will gain will not make you take action, then don't waste your time measuring it in the first place.

THE LEARNING OBJECTIVES of this lecture are that you will be able to:

- Understand the key role of distance and similarity in unsupervised learning and pattern recognition.
- Compute and interpret common distance metrics (Euclidean, Manhattan, Cosine, Mahalanobis).
- Apply appropriate normalization and scaling strategies and understand why they are necessary.

Distance and Similarity

QUITE INTUITIVELY, you already connected points and formed groups in terms of a similarity which was based on the closeness between data points. This concept will carry you very far.

A DISTANCE FUNCTION $d : X \times X \to \mathbb{R}_{\geq 0}$, which means taking two samples out of the set X, mapping them to a non-negative real number, must satisfy:

$$\text{Non-negativity:} \quad d(x,y) \geq 0 \tag{1}$$

$$\text{Identity:} \quad d(x,y) = 0 \iff x = y \tag{2}$$

$$\text{Symmetry:} \quad d(x,y) = d(y,x) \tag{3}$$

$$\text{Triangle inequality:} \quad d(x,z) \leq d(x,y) + d(y,z) \tag{4}$$

EUCLIDEAN DISTANCE is probably the most familiar distance metric, measuring the straight-line distance between two points in space.

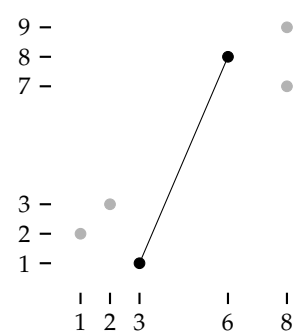

Figure 3: Euclidean distance between points (3,1) and (6,8).

And it was probably the one you implicitly used to connect the points in the previous exercise. It is defined as:

$$\begin{aligned} d_{\text{Euclidean}}(x,y) &= \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \\ &= \|x - y\|_2 \end{aligned}$$

Calculating the Euclidean distance between points $A = (3,1)$ and $B = (6,8)$:

$$\begin{aligned} d(A,B) &= \sqrt{(6-3)^2 + (8-1)^2} \\ &= \sqrt{9+49} \\ &= \sqrt{58} \qquad \approx 7.62 \end{aligned}$$

EUCLIDEAN DISTANCE assumes all dimensions are equally important and measured in the same units. It also assumes a continuous, smooth space. It is trivial to think of scenarios where these assumptions do not hold, such as when features have different scales (e.g. age vs. income) or when the features are not linearly related or not equaly important (e.g. size vs. IQ). And then there are the cases where the units are ambiguous for the task like distances on maps.

FEATURES can be thought of as dimensions in a vector space. Each data point is a vector in this space.

MANHATTAN DISTANCE is also called L_1 distance or taxicab distance. Instead of measuring the euclidean distance, it measures the axis-aligned path along grid lines:

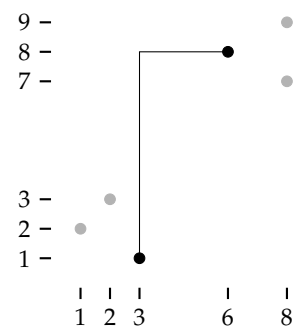

Figure 4: Manhattan (taxicab) distance between points (3,1) and (6,8): path follows grid lines.

$$d_{\text{Manhattan}}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
$$= \|x - y\|_1$$

Figure 5: Grid, 1811 (public domain) see Bridgeman Art Library v. Corel Corp.

ESPECIALLY USEFUL when movement is constrained to axes, such as for city blocks like the ones in Manhattan, pixel grids, or when you want robustness to outliers in individual dimensions.

Calculating the Manhattan distance between points $A = (3, 1)$ and $B = (6, 8)$:

$$d_M(A, B) = |6 - 3| + |8 - 1|$$
$$= 3 + 7$$
$$= 10$$

COSINE SIMILARITY measures the angle between two vectors, ignoring their magnitude. It is especially useful when the correlation of the features of the data matters more than its absolute values.

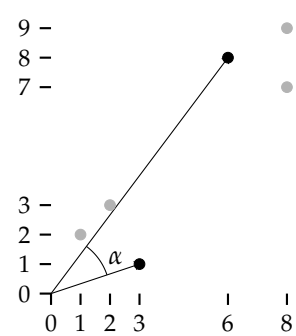

Figure 6: Cosine similarity between points (3,1) and (6,8): measuring the angle α between the position vectors.

When magnitude in the feature space does not matter only direction we use cosine similarity. It is defined as:

Calculating the cosine similarity between points $A = (3, 1)$ and $B = (6, 8)$:

$$\text{sim}(A, B) = \frac{3 \cdot 6 + 1 \cdot 8}{\sqrt{3^2+1^2}\,\sqrt{6^2+8^2}}$$
$$= \frac{26}{\sqrt{10} \cdot 10}$$
$$\approx 0.82$$

$$
\begin{aligned}
d_{\text{cosine}}(x, y) &= 1 - \frac{x \cdot y}{\|x\| \|y\|} \\
&= 1 - \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}
\end{aligned}
$$

The right-hand term is the cosine similarity, which ranges from -1 (opposite) to $+1$ (identical direction), so the distance ranges from 0 to 2.

THINK of a vector multiplication as a projection of one vector onto another.

MAGNITUDE IS IRRELEVANT for example, in text analysis, documents can have vectors of very different lengths as some documents have more words than others, but cosine similarity will still focuse on how many specific words per total word count. Similarly, in recommendation systems, users might have preference vectors with different scales, as some users rate more items, some less, but cosine similarity will compare the pattern of preferences.

MAHALANOBIS DISTANCE generalizes Euclidean distance by accounting for different variances and correlations among features. It measures distance in terms of the data's own covariance structure. In other words, it transforms the space so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods. The covariance matrix Σ captures the spread and correlation of the data, and is defined as:

$$
\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \mu_i)(x_i - \mu_i)^T
$$

where μ is the mean vector of the data.

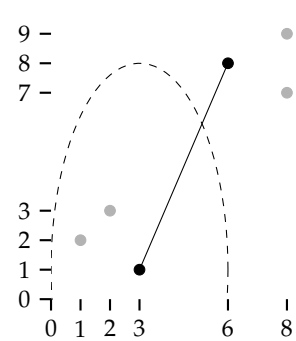

Figure 7: Mahalanobis distance between points (3,1) and (6,8): the dashed ellipse shows equidistant points under covariance Σ, replacing the Euclidean circle with an ellipse that reflects the data's spread.

Euclidean distance treats all dimensions equally, but real data often has different variances per dimension or correlations between

dimensions. The Mahalanobis distance compensates by weighting each dimension according to the data's spread. The inverse covariance matrix Σ^{-1} is used to weight the distance calculation, giving more weight to dimensions with less variance and less weight to dimensions with more variance.

$$d_{\text{Mahalanobis}}(x, y) = \sqrt{(x-y)^T \Sigma^{-1} (x-y)}$$

where Σ is the covariance matrix of the data.

Calculating the Mahalanobis distance between $A = (3, 1)$ and $B = (6, 8)$ with $\Sigma = \left(\begin{smallmatrix} 9 & 0 \\ 0 & 49 \end{smallmatrix}\right)$:

$$\begin{aligned} d_M(A, B) &= \sqrt{\frac{3^2}{9} + \frac{7^2}{49}} \\ &= \sqrt{1+1} \\ &= \sqrt{2} \\ &\approx 1.41 \end{aligned}$$

IT TRANSFORMS THE SPACE so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods.

DATA PREPROCESSING, before computing distances, ensures that features are comparable. You have already seen, how the Mahalanobis distance incorporates the covariance structure of the data to account for different scales and correlations. With the Euclidean distance, if features are on different scales, the distance will be dominated by the larger-scaled feature. If one feature's measure has a different magnitude than the other, the larger-scaled feature will dominate the distance calculation. Preprocessing brings all features to a common ground by transforming the feature space into a standardized scale.

NORMALIZATION typically refers to scaling features to a specific range, such as $[0, 1]$. While standardization refers to centering features to have mean 0 and scaling to have standard deviation 1.

Z-SCORE NORMALIZATION transforms each feature to have mean 0 and standard deviation 1. This centers the data at the origin and scales each dimension by its own spread:

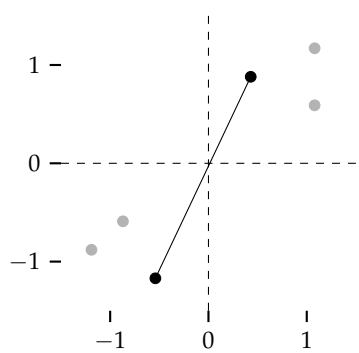

Figure 8: Our six points after z-score normalization: centered at the origin with unit variance in each dimension.

The z-score of each feature is computed as:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} \tag{5}$$

where μ_i and σ_i are the mean and standard deviation of feature i.

Normalizing point $A = (3, 1)$:

$$\begin{aligned} \mu &= (4.67,\ 5.00) \\ \sigma &= (3.08,\ 3.41) \\ z_x &= \frac{3 - 4.67}{3.08} = -0.54 \\ z_y &= \frac{1 - 5.00}{3.41} = -1.17 \\ A' &= (-0.54,\ -1.17) \end{aligned}$$

Use when features have different units or scales (e.g., age in years, vs. body height in milimiters). However, this is sensitive to outliers, as they influence μ and σ.

Min-Max Scaling transforms features to a fixed range $[0, 1]$. Each feature's minimum maps to 0 and its maximum to 1:

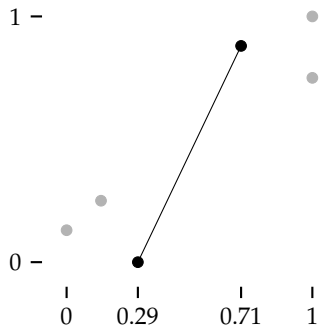

Figure 9: Our six points after min-max scaling: all values lie in $[0, 1]$.

$$x'_i = \frac{x_i - \min_i}{\max_i - \min_i} \tag{6}$$

Scaling point $A = (3, 1)$ with $x \in [1, 8]$, $y \in [1, 9]$:

$$x' = \frac{3-1}{8-1} = \frac{2}{7} \approx 0.29$$
$$y' = \frac{1-1}{9-1} = 0$$
$$A' = (0.29,\ 0)$$

Use when you need bounded values. Unlike z-score, min-max guarantees a fixed range but is even more sensitive to outliers since a single extreme value stretches the scale and compresses other ranges.

Choosing a Distance Metric encodes your assumptions about what makes two data points similar in a given context. The same pair of points can appear close or far apart depending on the metric. The choice should be driven by the structure of your data, the context you are in, and the question you are answering.

A practical rule of thumb: try Euclidean first as a baseline, then ask whether the assumptions it makes, equal scales, independent features, magnitude matters, actually hold for your data. Each violation points toward a more suitable metric.

Distance between $A{=}(3, 1)$ and $B{=}(6, 8)$ under each metric:

$$d_{\text{Euclidean}} = 7.62$$
$$d_{\text{Manhattan}} = 10$$
$$d_{\text{Cosine}} = 0.18$$
$$d_{\text{Mahalanobis}} = 1.41$$

Same points, four different answers. The distance defines the similarity.

Examples & Exercises

Exercises are for practice and reinforcing concepts. Try to solve them on your own first, try things, play with it, discuss, this is not a time trial. And there is no shame in not ending up at the right answer, in the same sense, that uncovering great questions and tossing them around is usually pretty fruitful on the long run.

$A = (2, 5, 1)$
$B = (6, 1, 3)$
$C = (4, 5, 2)$

Computing by hand builds the intuition that no amount of library calls can replace. Step through the arithmetic slowly.

Given points A, B, and C, compute the Euclidean distance $d_E(A, B)$, the Manhattan distance $d_M(A, C)$, and the cosine similarity $\text{sim}(A, B)$ with its corresponding cosine distance. If you feel you need more practice, go for it.

Euclidean distance sums squared coordinate differences:

$$\begin{aligned} d_E(A, B) &= \sqrt{(6-2)^2 + (1-5)^2 + (3-1)^2} \\ &= \sqrt{16 + 16 + 4} \\ &= \sqrt{36} \\ &= 6 \end{aligned}$$

Manhattan distance sums absolute differences instead, no squaring, no root:

$$\begin{aligned} d_M(A, C) &= |2-4| + |5-5| + |1-2| \\ &= 2 + 0 + 1 \\ &= 3 \end{aligned}$$

Cosine similarity measures the angle between the two vectors, independent of their length:

$$\begin{aligned} \text{sim}(A, B) &= \frac{A \cdot B}{\|A\| \, \|B\|} \\ &= \frac{2{\cdot}6 + 5{\cdot}1 + 1{\cdot}3}{\sqrt{4{+}25{+}1}\,\sqrt{36{+}1{+}9}} \\ &= \frac{20}{\sqrt{30}\,\sqrt{46}} \\ &\approx 0.54 \end{aligned}$$

The cosine distance follows as:

$$\begin{aligned} d_{\cos} &= 1 - 0.54 \\ &= 0.46 \end{aligned}$$

The metric is a modelling choice. There is no formula that tells you which distance to use, it follows from understanding your data and the context you are in. In the following scenarios, decide which distance metric is most appropriate and justify your choice. First

reason geometrically, then commit to an answer. Zoom-in so you can not see the margin notes, and try to solve it on your own first. Then read the margin notes for the answer.

A NEWS PLATFORM represents 10 000 articles as word-frequency vectors. Some articles are 100 words long, others 5 000 words. You want to find articles that cover similar topics.

Cosine similarity. Article length inflates the magnitude of the frequency vector but does not change its direction. Cosine similarity ignores magnitude, comparing only the distribution of words.

A DELIVERY COMPANY plans routes through Manhattan's grid. Pickup locations are given as (street, avenue) coordinates.

Manhattan distance. Movement on a grid is constrained to horizontal and vertical steps. The shortest driving path between two intersections follows the L_1 norm, not the straight line.

A COFFEE QUALITY LAB measures sourness (0–200) and bitterness (0–500) for each batch.

Mahalanobis distance. The features live on different scales and are correlated. Mahalanobis accounts for both by using the covariance matrix, producing ellipsoidal contours that align with the data's spread.

GIVEN THREE COFFEE BATCHES measured by sourness and bitterness:

$$
\begin{aligned}
P_1 &= (80,\ 400) \\
P_2 &= (200,\ 160) \\
P_3 &= (110,\ 220)
\end{aligned}
$$

compute the sample covariance matrix and use it to find the Mahalanobis distance between P_1 and P_3.

The mean vector:

$$
\begin{aligned}
\mu_x &= 130 \\
\mu_y &= 260 \\
\mu &= (130,\ 260)
\end{aligned}
$$

Deviations from the mean:

$$
\begin{aligned}
P_1 - \mu &= (-50,\ \ 140) \\
P_2 - \mu &= (\ \ 70,\ -100) \\
P_3 - \mu &= (-20,\ -40)
\end{aligned}
$$

The sample covariance matrix sums all three outer products:

$$
\begin{aligned}
\Sigma &= \frac{1}{N-1} \sum (x_i - \mu)(x_i - \mu)^T \\
&= \frac{1}{2} \begin{pmatrix} 7800 & -13200 \\ -13200 & 31200 \end{pmatrix} \\
&= \begin{pmatrix} 3900 & -6600 \\ -6600 & 15600 \end{pmatrix}
\end{aligned}
$$

We invert Σ analytically using the 2×2 formula:

$$\det(\Sigma) = 3900 \cdot 15600 - 6600^2 = 17{,}280{,}000$$

$$\Sigma^{-1} = \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}$$

For a 2×2 matrix:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A^{-1} = \frac{1}{ad - bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Swap the diagonal, negate the off-diagonal, divide by the determinant.

The difference vector is $\delta = P_3 - P_1 = (30,\ -180)$. Applying Σ^{-1}:

$$\Sigma^{-1}\delta = \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}\begin{pmatrix} 30 \\ -180 \end{pmatrix}$$

$$= \frac{1}{17{,}280{,}000}\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix}$$

INTUITIVELY, A^{-1} undoes the transformation A: if A stretches and rotates space, A^{-1} maps everything back, with $AA^{-1} = I$. For Σ, the inverse rescales each dimension by its variance and removes correlations, so distances are measured in standard-deviation units.

Taking the dot product with δ gives the squared distance:

$$\begin{aligned} d_M^2 &= \delta^T(\Sigma^{-1}\delta) \\ &= \frac{1}{17{,}280{,}000}(30,\ -180)\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix} \\ &= \frac{69{,}120{,}000}{17{,}280{,}000} = 4 \\ d_M &= \sqrt{4} = 2 \end{aligned}$$

Compare with the Euclidean distance, which does not account for the different scales and correlation. The raw displacement of 30 sourness units and 180 bitterness units yields an overall distance of 182.5, making P_3 seem very far from P_1, driven almost entirely by the bitterness difference:

$$\begin{aligned} d_E(P_1, P_3) &= \sqrt{(110-80)^2 + (220-400)^2} \\ &= \sqrt{900 + 32400} \\ &= \sqrt{33300} \\ &\approx 182.5 \end{aligned}$$

The Mahalanobis distance is far smaller because the covariance matrix rescales for the large variance difference between features. The bitterness axis has a much larger spread (σ^2=15600 vs σ^2=3900), so a displacement of 180 bitterness units is less unusual than it appears, and contributes less to the Mahalanobis distance accordingly.

STANDARDIZATION does not change the data, it changes the expanse and spread of the space you measure it in. Units disappear; and make way for a more abstract notion of distance, in terms of how many standard deviations apart the data points are.

$C_1 = (300,\ 7)$
$C_2 = (500,\ 6)$
$C_3 = (700,\ 3)$

daily cups sold, customer rating (1–10)

THE EFFECT OF STANDARDIZATION on café data. Three cafés C_1, C_2, C_3 recorded as daily cups sold and customer rating.

Compute the Euclidean distance on raw features, standardize all three points, and recompute. On raw features, daily sales dominate completely; the two pairs involving C_2 look equally close:

$$d_E(C_1, C_2) = \sqrt{(500-300)^2 + (6-7)^2} = \sqrt{40\,000 + 1} \approx 200.0$$
$$d_E(C_2, C_3) = \sqrt{(700-500)^2 + (3-6)^2} = \sqrt{40\,000 + 9} \approx 200.0$$

The rating differences (1 and 9) are drowned out by the cup counts (40,000 each). The means and standard deviations are:

$$\mu_{\text{cups}} = 500, \qquad \sigma_{\text{cups}} = 200$$
$$\mu_{\text{rating}} \approx 5.33, \qquad \sigma_{\text{rating}} \approx 2.08$$

After z-score normalization the cafés map to:

$$C_1' \approx (-1,\ 0.80), \qquad C_2' \approx (0,\ 0.32), \qquad C_3' \approx (1,\ -1.12)$$

THE CUPS standardize cleanly because $300, 500, 700$ are evenly spaced. The ratings do not: $7, 6, 3$ give a non-integer mean and an irrational standard deviation. This is what real data looks like.

Recomputing the distances:

$$d(C_1', C_2') = \sqrt{(0-(-1))^2 + (0.32-0.80)^2} = \sqrt{1 + 0.23} \approx 1.11$$
$$d(C_2', C_3') = \sqrt{(1-0)^2 + (-1.12-0.32)^2} = \sqrt{1 + 2.07} \approx 1.75$$

C_2 is now clearly closer to C_1 than to C_3. In raw space, sales made both pairs look identical; after standardization, the similar reviews of C_1 and C_2 pull them together while C_3's low rating pushes it away.

A TOY DATASET TO EXPLORE. You are given measurements of 20 coffee samples with three features: *acidity* (pH, 4.0–7.0), *bitterness* (1–10), and *brew strength* (mg/mL, 5–25). The samples belong to four varieties: cold brew, drip, espresso, and latte. Load the dataset and scatter-plot acidity vs. bitterness. Then compute the Euclidean distance matrix and identify the most similar pair, is the result plausible? Apply z-score normalization, recompute the distance matrix, and observe how the nearest neighbors shift. Compute cosine similarity between all pairs and compare it to the Euclidean grouping. Finally, compute the sample covariance matrix of the normalized data and check whether any features are correlated.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

20 coffee samples
acidity (pH, 4.0–7.0)
bitterness (1–10)
brew strength (mg/mL, 5–25)

four varieties: cold brew, drip, espresso, latte

WHAT TO OBSERVE. Without normalization, brew strength dominates similarity because its scale (5–25 mg/mL) dwarfs the others. A latte and an espresso with similar strength appear closer than two espressos with the same acidity and bitterness profile. After normalization, all three features contribute equally and the four natural

clusters become visible. Cosine similarity groups samples by the *ratio* of their features, which may merge varieties with similar flavor profiles but very different intensities.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during the exercises and afterwards:

- How do we distinguish between supervised and unsupervised learning?
- Elaborate on the differences between $\tilde{y}$, $\hat{y}$, and y?
- What are the key properties that define a distance metric?
- How do different distance metrics capture different notions of similarity?
- What is a feature space, and how do data points relate to it?
- What do we mean by feature engineering and feature transformation?
- What can happen when features are on different scales or magnitudes?
- How can we engineer and transform feature spaces?
- Why do we transform feature spaces with respect to pattern? And what are the common transformations?

RECAP of Key Concepts:

- Unsupervised learning discovers pattern in unlabeled data
- Distance metrics define what similar means
- Feature spaces can be engineered and transformed to reveal different patterns

SO FAR, WE DID MEASURE SIMILARITY BETWEEN SINGLE DATA POINTS. How do we embed this information across data points? Is it possible to represent pattern in the data as a whole, rather than just pairwise similarities? In the next chapter we explore clusters and clustering algorithms, which are the most classic way to represent pattern in unsupervised learning.

WITHOUT KNOWING ABOUT LABELS OR SEMANTICS we can tell which samples are similar.

TEASER. How do we represent structure beyond pairwise similarities and distances?

FEEDBACK

Centroid Clustering

From Similar to of the Same

The Why

In the previous chapter, we established that distance is a choice that is to be engineered[4]: it encodes what similar means in a given feature space. But similarity is a local, pairwise concept so far.

As a human looking at data, we immediately see clusters of points that belong together, and we have an intuition for which new points would fit into which cluster. The process of finding them is called clustering.

[4] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

A partition or a clustering assigns every point to exactly one cluster, with no cluster left empty. In this lecture we will learn about K-Means[5], which produces a *hard* partition: membership is binary, not graded.

[5] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967. https://scholar.google.com/scholar?q=MacQueen+1967+methods+classification+multivariate+observations

Given that means to encode similarity over whole sets of samples, we can ask a more complex question, one that goes beyond pairs of points. Being able to answer which point belongs to which cluster is a predictive power that allows us to generalize from observed data to new instances.

Binary membership is a strong assumption that later clustering methods will soften.

In order to move from representation to leveraging pattern we have good reason to find ways to,

- compute partitions, of the data, given a distance measure.
- formalize what makes a well-defined partition by specifying an appropriate objective.
- recognize the geometric assumptions and finding ways to represent cluster characteristics.

Figure 10: Clustering allows us to move from a) and b) are similar to a) and b) are of the same. *Image is generated with NanoBanana.*

Hands On Experience

CONSIDER twelve data points in two dimensions. Which points belong together? How many clusters do you see?

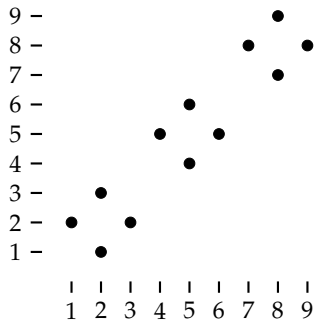

Figure 11: Twelve points in 2D. How many clusters do you see?

HOW MANY CLUSTERS do you see? Three seems natural, but could you argue for two? For six? For 12?

WITHOUT GROUND TRUTH, there is no single right answer, as a context for any answer could be engineered. However, some answers are better than others, and it does not come as a surprise when I tell you that most people see three clusters in this data.

THE FUNDAMENTAL QUESTION this chapter answers is how do we find the clusters mathematically? We will see how K-Means assigns each point to a cluster; and how cluster centroids function as representative points for each cluster.

WHERE WOULD YOU PLACE a representative point for each cluster? Could you think of other measures to characterize and represent clusters, beyond a single point?

A CENTROID is the mean position of all points assigned to a cluster. It need not be an actual data point; it is the center of mass of the cluster. The centroid exists in the same space as the data. With a different distance, a different notion of center is needed. For sake of simplicity and interpretability, we tie ourselves to Euclidean distance and arithmetic means. But it is worth getting an intuition for other distances as well.

THE LEARNING OBJECTIVES of this lecture:

- Understand the K-Means objective and algorithm, including convergence.
- Apply initialization strategies and K-selection methods.
- Identify when K-Means fails and understand why.

The K-Means Algorithm

A DATA POINT is a vector $x_i \in \mathbb{R}^d$, where d is the number of features and $i = 1, \ldots, n$ indexes the observations.

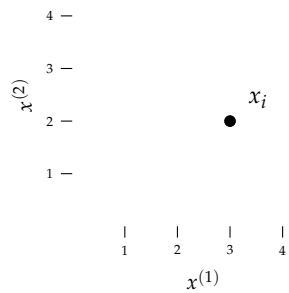

Figure 12: A single data point x_i in $\mathbb{R}^2$.

A CLUSTER is a non-empty subset $C_k \subseteq \{x_1, \ldots, x_n\}$. Each point belongs to exactly one cluster; this is the hard assignment assumption, meaning clusters are disjoint:

$$C_j \cap C_k = \emptyset \quad \text{for } j \neq k$$

A clustering of the data is a collection of K clusters $\{C_1, \ldots, C_K\}$ that is exhaustive, every point is accounted for:

$$\bigcup_{k=1}^{K} C_k = \{x_1, \ldots, x_n\}$$

K-MEANS finds a clustering of n data points into K clusters by minimizing the total squared distance from each point to its cluster's centroid. This quantity is called inertia or within-cluster sum of squares (WCSS):

MINIMIZING J is NP-hard (effort exponential in n and K) in general. K-Means finds a local optimum, not necessarily the global one. The result depends on initialization, and is thus non-deterministic. Shout out numerical methods.

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \tag{7}$$

where the mean $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ is the centroid of cluster C_k.

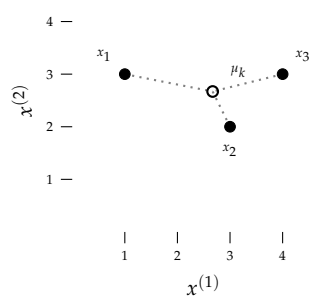

Figure 13: Cluster $C_k = \{x_1, x_2, x_3\}$ with centroid μ_k. Dotted lines show the distances whose squares sum to J.

THE ALGORITHM ALTERNATES BETWEEN TWO STEPS until the assignments no longer change. It remains one of the most cited algorithms in machine learning, in part because it is easy to implement, easy to interpret, and fast enough to run on large datasets.

Algorithm 1: K-Means

Require: Data $X = \{x_1, \ldots, x_n\}$, number of clusters K, initial centroids $\mu_1, \ldots, \mu_K$
Ensure: Clustering $\{C_1, \ldots, C_K\}$, centroids $\{\mu_1, \ldots, \mu_K\}$

```
repeat
    Assignment: c_i ← arg min_k ||x_i − μ_k||^2 for each x_i
    Update: μ_k ← (1/|C_k|) Σ_{x_i : c_i = k} x_i for each k
until assignments no longer change
```

STANDARD ALGORITHMIC FORM was described by Lloyd

S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. DOI: 10.1109/TIT.1982.1056489. https://scholar.google.com/scholar?q=Lloyd+1982+least+squares+quantization+PCM

A TRACE THROUGH THE ALGORITHM on three points with K=2 makes this concrete. With initial centroids at μ_1=x_1=(1,3) and μ_2=x_3=(4,3):

- For each point, the assignment step asks: which centroid is closest?
- For each cluster, the update step asks: given the current assignments, where is the new centroid?

STEP 1: INITIALIZATION. We place two centroids at $\mu_1 = (1,4)$ and $\mu_2 = (4,1)$, away from any data point.

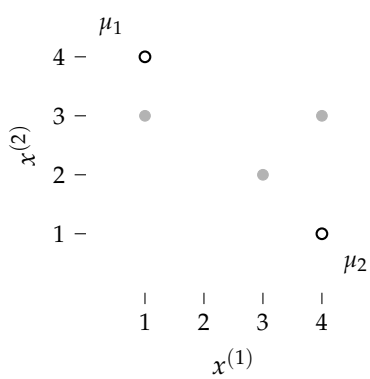

Figure 14: Initialization: two centroids placed away from the data. All points are unassigned.

STEP 2: ASSIGNMENT. Each point joins the cluster of its nearest centroid.

$$
\begin{aligned}
x_1{=}(1,3): &\quad d^2(\mu_1) = 0+1 = 1, \quad d^2(\mu_2) = 9+4 = 13 \;\rightarrow\; C_1 \\
x_2{=}(3,2): &\quad d^2(\mu_1) = 4+4 = 8, \quad d^2(\mu_2) = 1+1 = 2 \;\rightarrow\; C_2 \\
x_3{=}(4,3): &\quad d^2(\mu_1) = 9+1 = 10, \quad d^2(\mu_2) = 0+4 = 4 \;\rightarrow\; C_2
\end{aligned}
$$

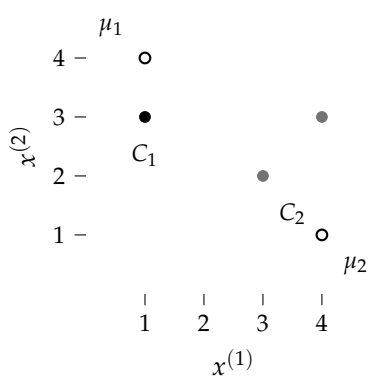

Figure 15: Assignment: $x_1 \rightarrow C_1$ (dark), x_2 and $x_3 \rightarrow C_2$ (grey).

STEP 3: UPDATE. Each centroid moves to the mean of its cluster.

$$\mu_1' = x_1 = (1,\ 3)$$
$$\mu_2' = \tfrac{1}{2}\big((3,2) + (4,3)\big) = (3.5,\ 2.5)$$

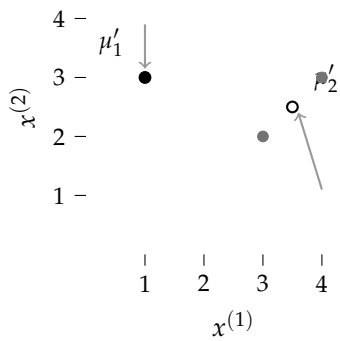

Figure 16: Update: both centroids move (arrows). μ_1 drops from $(1,4)$ to $(1,3)$; μ_2 jumps from $(4,1)$ to $(3.5,2.5)$. Re-running assignment yields the same clusters: converged.

CONVERGENCE is guaranteed in finite steps. The argument rests on three observations: first, each assignment step can only decrease J, since every point moves to an equal or closer centroid; second, each update step can only decrease J, since the mean minimizes squared distance to its members; third, there are at most K^n possible assignment configurations. Since J is non-increasing and the number of states is finite, the algorithm must reach a fixed assignment and stop.

Typical convergence takes 10–100 iterations in practice.

LOCAL OPTIMUM. Convergence does not mean the algorithm found the best clustering. It found a clustering. A different initialization may yield a better one.

INITIALIZATION MATTERS

K-Means is sensitive to where centroids start. If two initial centroids fall inside the same natural cluster, one cluster gets split and another gets merged, and the algorithm may never recover.

MULTIPLE RESTARTS. A practical fix: run K-Means n times with different random initializations, keep the result with the lowest J. Typical default: n=10.

K-MEANS++[6] gives a smarter starting point. The first centroid is chosen uniformly at random. Each subsequent centroid is chosen with probability proportional to its squared distance to the nearest already-chosen centroid:

$$p(x_i) = \frac{D(x_i)^2}{\sum_j D(x_j)^2}$$

where $D(x_i) = \min_{j<k} \|x_i - \mu_j\|$. Points far from all current centroids are more likely to be chosen next, spreading the initial

[6] D. Arthur and S. Vassilvitskii. K-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007. https://scholar.google.com/scholar?q=Arthur+Vassilvitskii+2007+k-means+advantages+careful+seeding

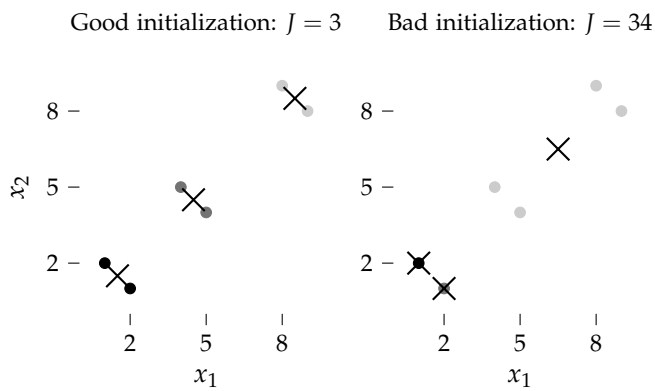

Figure 17: Same data, different initial centroids. Left: one centroid per natural cluster gives J=3. Right: two centroids inside the same cluster leave the other two clusters merged, giving J=34.

centroids across the data. This gives a theoretical guarantee on the expected inertia relative to the global optimum.

Choosing K

K must be set by the practitioner. Two complementary heuristics help.

The elbow method plots inertia J against K and looks for a kink: the point where adding one more cluster yields diminishing returns. For our six points, J drops sharply from K=1 to K=3, then flattens.

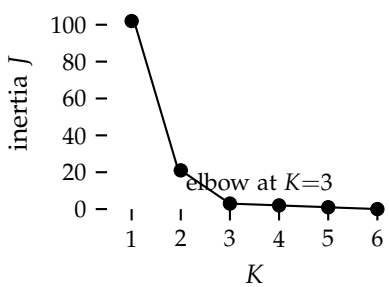

Figure 18: Elbow plot for the six sample points. The kink at K=3 matches the three natural clusters.

The elbow is not always clear. On real data, the curve may decrease smoothly with no obvious kink. The elbow is a heuristic, not a guarantee.

The silhouette score measures how well each point fits its assigned cluster relative to the next-best cluster. For point x_i, let $a(i)$ be its mean distance to points in the same cluster and $b(i)$ its mean distance to points in the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}$$

$s(i)$ ranges from -1 (point would fit better elsewhere) to $+1$ (point

is well-placed). The mean silhouette over all points is a quality score for the full clustering. Choose K that maximizes it.

For our six points with K=3, every point has $a(i)=\sqrt{2} \approx 1.4$ and $b(i) \approx 4.4$, giving $s(i) \approx 0.68$. A clear, well-separated clustering.

When K-Means Fails

K-Means assumes clusters are roughly spherical, equally sized, and equally dense. When the data violates these assumptions, the Voronoi partition imposed by Euclidean centroids cuts the data in the wrong places.

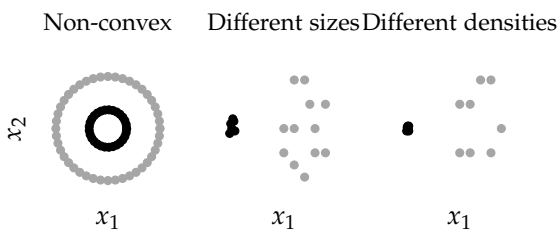

Figure 19: Three scenarios where K-Means fails. Left: concentric rings share a centroid and cannot be separated. Centre: a large and a small cluster pull the boundary toward the small one. Right: a dense and a sparse cluster of equal inertia look equivalent to K-Means.

When K-Means struggles, the underlying issue is always the same: the Euclidean distance to a centroid is the wrong measure of belonging for that data. The next chapters address this by changing the notion of neighborhood (hierarchical clustering) and of density (DBSCAN).

K-Medoids uses actual data points as cluster centers instead of means, making it robust to outliers and compatible with non-Euclidean distances.

Mini-Batch K-Means updates centroids on random subsets of the data, scaling to millions of points at the cost of a slightly noisier result.

Examples & Exercises

Exercises are for practice and reinforcing concepts. Try to solve them on your own first, try things, play with it, discuss; this is not a time trial. And there is no shame in not ending up at the right answer, in the same sense that uncovering great questions and tossing them around is usually pretty fruitful in the long run.

$A = (1, 2)$, $B = (2, 1)$
$C = (4, 5)$, $D = (5, 4)$
$E = (8, 9)$, $F = (9, 8)$

acidity, bitterness (1–10)

Computing by hand builds the intuition that no amount of library calls can replace. Step through the arithmetic slowly.

Given six coffee samples A–F, run K-Means with K=3 and initial centroids μ_1=A, μ_2=C, μ_3=E. Perform the assignment step, the update step, and compute the inertia after convergence.

The assignment step computes the squared distance from each point to each centroid and assigns each point to the nearest one. The distances are symmetric by construction so we only check whether a point is closest to its natural cluster center:

$$
\begin{aligned}
d(A,\mu_1)^2 &= 0, \quad d(A,\mu_2)^2 = 18, \quad d(A,\mu_3)^2 = 98 \;\rightarrow\; C_1 \\
d(B,\mu_1)^2 &= 2, \quad d(B,\mu_2)^2 = 20, \quad d(B,\mu_3)^2 = 100 \;\rightarrow\; C_1 \\
d(C,\mu_1)^2 &= 18, \quad d(C,\mu_2)^2 = 0, \quad d(C,\mu_3)^2 = 32 \;\rightarrow\; C_2 \\
d(D,\mu_1)^2 &= 20, \quad d(D,\mu_2)^2 = 2, \quad d(D,\mu_3)^2 = 34 \;\rightarrow\; C_2 \\
d(E,\mu_1)^2 &= 98, \quad d(E,\mu_2)^2 = 32, \quad d(E,\mu_3)^2 = 0 \;\rightarrow\; C_3 \\
d(F,\mu_1)^2 &= 100, \quad d(F,\mu_2)^2 = 34, \quad d(F,\mu_3)^2 = 2 \;\rightarrow\; C_3
\end{aligned}
$$

The update step recomputes each centroid as the mean of its assigned points:

$$
\begin{aligned}
\mu_1' &= \tfrac{1}{2}\big((1,2) + (2,1)\big) = (1.5,\ 1.5) \\
\mu_2' &= \tfrac{1}{2}\big((4,5) + (5,4)\big) = (4.5,\ 4.5) \\
\mu_3' &= \tfrac{1}{2}\big((8,9) + (9,8)\big) = (8.5,\ 8.5)
\end{aligned}
$$

Running the assignment step again with the new centroids yields the same clusters; the algorithm has converged. The inertia is:

$$
\begin{aligned}
J &= \|A - \mu_1'\|^2 + \|B - \mu_1'\|^2 + \|C - \mu_2'\|^2 + \|D - \mu_2'\|^2 + \|E - \mu_3'\|^2 + \|F - \mu_3'\|^2 \\
&= 0.5 + 0.5 + 0.5 + 0.5 + 0.5 + 0.5 \\
&= 3
\end{aligned}
$$

Each pair contributes equally; the clustering is geometrically symmetric.

NOW REPEAT with bad initial centroids $\mu_1{=}A$, $\mu_2{=}B$, $\mu_3{=}C$ — two centroids inside the cold-brew cluster.

Assignment step with $\mu_1{=}(1,2)$, $\mu_2{=}(2,1)$, $\mu_3{=}(4,5)$:

$$\begin{aligned} A &\to C_1 \quad \text{(distance 0 to } \mu_1\text{)} \\ B &\to C_2 \quad \text{(distance 0 to } \mu_2\text{)} \\ C,\, D,\, E,\, F &\to C_3 \quad \text{(all closer to } \mu_3 \text{ than to } \mu_1 \text{ or } \mu_2\text{)} \end{aligned}$$

WHAT GOES WRONG. Both μ_1 and μ_2 compete for cold-brew samples. The algorithm optimizes what it is given; it has no way of knowing that A and B are in the same natural cluster.

After the update: $\mu_1{=}(1,2)$ (singleton), $\mu_2{=}(2,1)$ (singleton), $\mu_3{=}(6.5, 6.5)$ (four points merged). The assignments do not change in the next iteration. The inertia of this converged solution is:

$$\begin{aligned} J_{\text{bad}} &= 0 + 0 + \|C - \mu_3\|^2 + \|D - \mu_3\|^2 + \|E - \mu_3\|^2 + \|F - \mu_3\|^2 \\ &= 0 + 0 + 8.5 + 8.5 + 8.5 + 8.5 \;=\; 34 \end{aligned}$$

The bad initialization found a solution more than ten times worse than the good one ($J{=}34$ vs $J{=}3$). This is why K-Means++ and multiple restarts matter.

FOR THE ELBOW METHOD, run K-Means on the same six points for $K = 1, \ldots, 6$ and record the inertia.

The drop from $K{=}1$ to $K{=}3$ is large (102 to 3); from $K{=}3$ onward the curve is nearly flat. The elbow is at $K{=}3$, matching the three natural coffee varieties. Note that $K{=}6$ always achieves $J{=}0$ by placing one centroid per point; this is not useful.

Inertia values:

K	J
1	102
2	21
3	3
4	2
5	1
6	0

A COFFEE DATASET TO EXPLORE. Load the dataset and scatter-plot acidity vs. bitterness. Run K-Means with $K{=}3$ on raw features and visualize the cluster assignments. Try different random seeds; how stable are the results? Then vary K from 1 to 8, record the inertia and silhouette score for each, and plot both curves. Finally, deliberately give K-Means a bad initialization and compare the resulting inertia with the K-Means++ result.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

$n = 20$ samples
acidity (1–10)
bitterness (1–10)

Varieties: cold brew, drip, espresso, latte.

WHAT TO OBSERVE. With a good initialization and $K{=}3$, the three clear clusters emerge immediately. The elbow and the silhouette peak agree at $K{=}3$. With a bad initialization, inertia is higher and cluster boundaries shift; the silhouette score drops. With $K{=}4$ a natural cluster splits in two, and the silhouette score begins to fall.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the K-Means objective actually minimize, and why is that a reasonable definition of a good clustering?
- Why does K-Means always converge, and why does convergence not guarantee the best solution?
- What does K-Means++ change about the initialization, and why does spreading centroids out help?
- When would you trust the elbow method? When would you not?
- How does the silhouette score measure something different from inertia?
- What shape of cluster cannot K-Means find, and why?
- How does the choice of distance metric interact with the K-Means update step?
- Given a new dataset, what would be your first three steps before running K-Means?

RECAP of Key Concepts:

- K-Means minimizes inertia by alternating assignment (points to nearest centroid) and update (centroid to cluster mean)
- Convergence is guaranteed; optimality is not, use K-Means++ or multiple restarts
- The elbow and silhouette score are complementary heuristics for choosing K
- K-Means assumes spherical, equally sized clusters; it fails on rings, elongated shapes, and density differences

TEASER. K-Means assigns every point to exactly one flat cluster. What if structure exists at multiple scales, or if some points genuinely belong between clusters?

SO FAR, WE FOUND FLAT PARTITIONS of the data into K fixed clusters. But natural structure is often hierarchical: coffee varieties cluster by roast, roasts cluster by origin, origins cluster by continent. And some points genuinely sit at the boundary of two clusters, which a hard assignment hides. The next chapters address both: hierarchical clustering reveals nested structure through dendrograms, and density-based methods find clusters of arbitrary shape without requiring a fixed K.

Index

Centroid Clustering, 23
centroid clustering, 25
cluster, 27
clustering, 27
Cosine similarity, 15
covariance matrix, 16

data point, 27
data preprocessing, 17
distance, 11
distance metric, 14
Distance Metrics, 11

elbow method, 30
Euclidean distance, 14
examples, 19, 32
exercises, 19, 32

failure modes, 31
feature space, 14
Foundations, 11

inertia, 27

K-Means, 23, 25, 27
K-Means++, 29

L1 distance, 15
license, 2

Mahalanobis distance, 16
Manhattan distance, 15
Min-Max scaling, 18

normalization, 17

recap, 23, 33
reflection, 23, 33

silhouette score, 30
similarity, 11
standardization, 17

unsupervised learning, 11

Z-score normalization, 17